

Week 8: Advanced Features — Partitioning, JSONB, FTS, and Extensions

Phase 3: Internals & Operations | Week 8 of 12

Week Overview

This week covers PostgreSQL's most powerful advanced features that enable it to compete with specialized databases. You will master table partitioning for massive datasets, full-text search, advanced JSONB patterns, and the extension ecosystem. These skills are expected at senior levels at data-intensive companies.

Focus: Partitioning and full-text search are the two most commonly underused PostgreSQL features. Learn them deeply.

Learning Objectives

By the end of this week, you will be able to:

- Implement RANGE, LIST, and HASH partitioning strategies.
 - Write partition pruning-friendly queries.
 - Add and detach partitions without downtime.
 - Build a full-text search system with `tsvector`, `tsquery`, and rankings.
 - Configure custom text search dictionaries.
 - Use `pg_trgm` for fuzzy search and LIKE performance.
 - Understand advanced JSONB patterns: path queries, aggregation, indexing.
 - Use key extensions: PostGIS, `pg_cron`, `pg_stat_statements`, `timescaledb`.
-

Required Reading

- `11_Advanced_PostgreSQL/` — All files
-

Daily Schedule

Monday — Table Partitioning (60 min)

Topics:

- Why partition: manageability, query pruning, parallel scans
- RANGE partitioning (dates, numeric ranges)
- LIST partitioning (regions, categories)
- HASH partitioning (even distribution)
- Partition pruning: how the planner avoids scanning all partitions
- Adding, detaching, and archiving partitions
- Global vs. local indexes on partitions

```
-- RANGE partitioning by month
CREATE TABLE events (
    event_id    BIGSERIAL,
```

```

    user_id    INTEGER NOT NULL,
    event_name VARCHAR(200) NOT NULL,
    payload    JSONB,
    created_at TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2024_10 PARTITION OF events
    FOR VALUES FROM ('2024-10-01') TO ('2024-11-01');
CREATE TABLE events_2024_11 PARTITION OF events
    FOR VALUES FROM ('2024-11-01') TO ('2024-12-01');
CREATE TABLE events_default PARTITION OF events DEFAULT;

-- Verify partition pruning
EXPLAIN SELECT * FROM events WHERE created_at >= '2024-10-01' AND created_at <
'2024-11-01';
-- Should show ONLY events_2024_10, not others

-- LIST partitioning by status
CREATE TABLE orders_part (
    id        BIGSERIAL,
    status    VARCHAR(20),
    amount    NUMERIC(12,2)
) PARTITION BY LIST (status);

CREATE TABLE orders_active PARTITION OF orders_part FOR VALUES IN
('pending','processing');
CREATE TABLE orders_closed PARTITION OF orders_part FOR VALUES IN
('completed','cancelled');
CREATE TABLE orders_default PARTITION OF orders_part DEFAULT;

-- HASH partitioning for even distribution
CREATE TABLE sessions (
    session_id VARCHAR(100),
    user_id    INTEGER
) PARTITION BY HASH (user_id);

CREATE TABLE sessions_p0 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER
0);
CREATE TABLE sessions_p1 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER
1);
CREATE TABLE sessions_p2 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER
2);
CREATE TABLE sessions_p3 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER
3);

-- Attach/detach for archiving
-- ALTER TABLE events DETACH PARTITION events_2024_10;
-- Now events_2024_10 is a standalone table – can be dumped/archived

```

Topics:

- `tsvector` : preprocessed document representation
- `tsquery` : search query with AND (&), OR (|), NOT (!)
- `to_tsvector` , `to_tsquery` , `plainto_tsquery` , `websearch_to_tsquery`
- `ts_rank` and `ts_rank_cd` for relevance scoring
- `ts_headline` for snippet generation
- Stored `tsvector` column with trigger update
- GIN index on `tsvector` column

```
-- Basic FTS
SELECT to_tsvector('english', 'PostgreSQL is an advanced open-source database');
-- Returns: 'advanced':4 'databas':7 'open-sourc':6 'postgresql':1

SELECT to_tsquery('english', 'postgresql & database');
-- Returns: 'postgresql' & 'databas'

-- Find matching documents
SELECT title FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ to_tsquery('english',
'PostgreSQL & index');

-- Create articles table with stored tsvector (best practice)
CREATE TABLE articles (
    id          SERIAL PRIMARY KEY,
    title       VARCHAR(400) NOT NULL,
    body        TEXT NOT NULL,
    author      VARCHAR(200),
    published_at DATE,
    search_vector TSVECTOR
);

-- GIN index for fast search
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vector);

-- Trigger to auto-update search_vector
CREATE OR REPLACE FUNCTION update_search_vector()
RETURNS TRIGGER AS $$
BEGIN
    NEW.search_vector := to_tsvector('english',
        COALESCE(NEW.title, '') || ' ' || COALESCE(NEW.body, ''))
);
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_articles_fts
BEFORE INSERT OR UPDATE OF title, body ON articles
FOR EACH ROW EXECUTE FUNCTION update_search_vector();

-- Insert test data
```

```

INSERT INTO articles (title, body, author, published_at) VALUES
  ('Understanding PostgreSQL MVCC', 'Multi-version concurrency control allows
  readers and writers to never block each other...', 'Alice', '2024-01-15'),
  ('Index Optimization Techniques', 'B-Tree, GIN, and GiST indexes serve different
  query patterns...', 'Bob', '2024-02-20'),
  ('Advanced Window Functions', 'Window functions compute values across a set of
  rows related to the current row...', 'Carol', '2024-03-10');

-- Search with ranking
SELECT
  title,
  ts_rank(search_vector, query) AS rank,
  ts_headline('english', body, query, 'MaxWords=30, MinWords=15') AS snippet
FROM articles,
  to_tsquery('english', 'index & optimization') AS query
WHERE search_vector @@ query
ORDER BY rank DESC;

-- Phrase search (exact phrase)
SELECT title FROM articles
WHERE search_vector @@ to_tsquery('english', 'window <-> function');

-- websearch_to_tsquery (accepts natural language)
SELECT title FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', 'index optimization
  techniques');

```

Wednesday — pg_trgm and Fuzzy Search (90 min)

Topics:

- Trigram similarity for fuzzy matching
- `similarity()` function (0-1 score)
- `%` similarity operator
- `<->` distance operator
- GIN and GIST trgm indexes for LIKE/ILIKE performance

```

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Similarity scoring
SELECT similarity('PostgreSQL', 'PotgreSQL'); -- typo! still ~0.65

-- % operator: true if similarity > threshold (default 0.3)
SELECT 'PostgreSQL' % 'PotgreSQL'; -- true

-- Order by similarity (most similar first)
SELECT name, similarity(name, 'postgress') AS sim
FROM articles_authors
ORDER BY sim DESC
LIMIT 5;

```

```

-- GIN trigram index enables fast LIKE/ILIKE
CREATE INDEX idx_articles_title_trgm ON articles USING GIN (title gin_trgm_ops);

-- Now ILIKE uses the index instead of seq scan
EXPLAIN SELECT * FROM articles WHERE title ILIKE '%window%';
-- Should use index

-- Distance operator for search-as-you-type
SELECT title, title <-> 'MVCC concurrency' AS distance
FROM articles
ORDER BY distance
LIMIT 5;

```

Thursday — Key Extensions (60 min)

Topics:

- `pg_stat_statements` : track all query execution stats
- `pg_cron` : schedule jobs inside PostgreSQL
- `uuid-oss` : UUID generation functions
- `tablefunc` : crosstab (pivot) queries
- `hstore` : key-value pairs (mostly superseded by JSONB)
- `ltree` : hierarchical label trees
- Introduction to TimescaleDB for time-series

```

-- pg_stat_statements: find your slowest queries
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT query, calls, mean_exec_time, total_exec_time, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

-- pg_cron: schedule jobs
CREATE EXTENSION IF NOT EXISTS pg_cron;

SELECT cron.schedule(
    'nightly-vacuum',
    '0 3 * * *', -- 3am every day
    'VACUUM ANALYZE orders'
);

SELECT cron.schedule(
    'hourly-report-refresh',
    '0 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_summary'
);

SELECT * FROM cron.job;

-- tablefunc: crosstab pivot

```

```

CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT * FROM crosstab(
    'SELECT department, metric, value FROM metrics_table ORDER BY 1,2',
    'SELECT DISTINCT metric FROM metrics_table ORDER BY 1'
) AS ct (department TEXT, headcount INTEGER, avg_salary NUMERIC);

-- ltree: hierarchical path queries
CREATE EXTENSION IF NOT EXISTS ltree;

CREATE TABLE category_tree (
    id      SERIAL PRIMARY KEY,
    path    LTREE NOT NULL,
    name    VARCHAR(100)
);

INSERT INTO category_tree (path, name) VALUES
    ('Electronics',          'Electronics'),
    ('Electronics.Computers', 'Computers'),
    ('Electronics.Computers.Laptops', 'Laptops'),
    ('Electronics.Phones',      'Phones');

-- All descendants of Electronics
SELECT * FROM category_tree WHERE path <@ 'Electronics';

-- Parent of Laptops
SELECT * FROM category_tree WHERE path @> 'Electronics.Computers.Laptops';

```

Friday — Mini-Project: Full-Text Search Engine (45 min)

Mini-Project: Build a document search system.

```

-- Requirements:
-- 1. Documents table (title, body, author, published_at, tags TEXT[])
-- 2. Stored tsvector with weighted columns (title is more important than body)
-- 3. GIN index + trgm index on title
-- 4. Search function that accepts a query string and returns ranked results with
snippets
-- 5. Tag filtering combined with FTS
-- 6. Pagination with consistent ranking

-- Bonus: setweight() to give title higher weight than body
-- A: title (weight 1.0), B: body (weight 0.4)
tsvector := setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', body), 'B');

```

Practice Tasks

1. Create a date-partitioned events table, insert data spanning 3 months, verify partition pruning with EXPLAIN.

2. Write a full-text search query that ranks results by relevance and returns highlighted snippets.
 3. Use `pg_trgm` to build an autocomplete-style search on product names.
 4. Schedule a `pg_cron` job that refreshes a materialized view every hour.
 5. Use `pg_stat_statements` to find the top 3 slowest queries in your test database.
 6. Implement weighted FTS (title > subtitle > body) using `setweight` .
 7. Detach an old partition, verify the parent table no longer shows its data, then re-attach.
 8. Create a `ltree` hierarchy and write a query to find all descendants at depth 2 or more.
-

Self-Assessment Checklist

- ☐ I can create range, list, and hash partitions
 - ☐ I verified partition pruning with EXPLAIN
 - ☐ I built a FTS system with stored tsvector and GIN index
 - ☐ I know the difference between `to_tsquery` and `websearch_to_tsquery`
 - ☐ I used `ts_headline` to generate search result snippets
 - ☐ I installed and used `pg_trgm` for fuzzy search
 - ☐ I used `pg_stat_statements` to identify a slow query
-

Mock Interview Questions

1. When would you use range partitioning vs. list partitioning?
 2. What is partition pruning and how does the query planner achieve it?
 3. Explain the difference between `tsvector` and `tsquery` .
 4. How does `pg_trgm` enable fast ILIKE queries?
 5. How would you build a real-time autocomplete search using PostgreSQL?
 6. What is the performance difference between searching JSONB with `->>` vs. GIN containment?
 7. How do you partition an existing table without downtime?
 8. What are the limitations of table partitioning in PostgreSQL?
-

Resources

- This repo: `11_Advanced_PostgreSQL/`
- PostgreSQL FTS: <https://www.postgresql.org/docs/16/textsearch.html>
- `pg_partman`: automated partition management extension